

1 P4 — Disjoint congruence classes: final write-up

Claimed cells: C1–C4 solved; C5 solved for the explicitly claimed contiguous boundary 14. C6 is not attempted. No platform submission or organizer acceptance is claimed.

Evidence distinction: the general theorem for every $2 \leq k \leq 12$ is proved in Lean. The written finite reduction and exhaustive, independently replayed computations extend the mathematical certificate through 16. The additional C5 rule-on/rule-off benchmark is completed only through 14, so 15 and 16 are outside the C5 boundary claim.

All linked proof artifacts below are pinned to the published checkpoint `ed55f76eb8c558825bb629fd2cd8dbf46e5f1314`. They contain complete source and exact certificate data, not merely references to an unpublished computation. This write-up assembles the arguments and gives the verification commands; it makes no novelty claim for established results.

1.1 Definitions and verification

Write $a \pmod m$ for $\{a + mt : t \in \mathbb{Z}\}$, with $m \geq 1$. Moduli and residues may repeat. Two classes intersect exactly when $\gcd(m_i, m_j)$ divides $a_i - a_j$. The statement at size k says some pair in every pairwise disjoint k -class family has modulus gcd at least k .

The bound is sharp: residues $0, 1, \dots, k-1$ modulo k are pairwise disjoint and all modulus gcds equal k .

Clone the repository and check out the pinned checkpoint above. Python uses only its standard library; Lean is pinned to 4.34.1 and uses bundled Std, with no Mathlib dependency.

```
# Full C4 computational replay, including both different enumeration methods.
```

```
python3 -B cagent/p4/audit/replay.py --max-k 16
```

```
# Complete C5 benchmark for the claimed boundary 14.
```

```
python3 -B cagent/p4/audit/replay.py --max-k 14
```

```
# Independent formal range proof; no Python output is trusted by Lean.
```

```
cd cagent/p4/lean
```

```
lake build ProofPursuit.P4.ThroughTwelve
```

```
lake env leanchecker ProofPursuit.P4.ThroughTwelve
```

The complete C4 Python replay took **500.013 seconds** on the recorded laptop run, below ten minutes. The C5 boundary-14 replay took **16.109 seconds**, and an independent integration replay took **16.387 seconds**. These are measured runtimes, not guarantees for every machine. Node counts and exact survivor lists reproduce; wall clocks need not be identical. The runner writes to a fresh temporary directory by default.

1.2 C1 — Three classes

Suppose three pairwise disjoint classes had every pair gcd below 3. A pair gcd cannot be 1 by the intersection criterion, so all three gcds would equal 2. Disjointness would require the three residues to be pairwise distinct modulo 2, impossible. Therefore some pair has gcd at least 3.

The complete formal proof is [C1.lean](#), using the proved CRT/Bézout infrastructure in [Basic.lean](#).

1.3 C2 — Four classes

Suppose all six pair gcds are below 4. Each is therefore 2 or 3.

If every modulus is even, each pair gcd must be 2. Already three residues would have to be pairwise distinct modulo 2, impossible.

Otherwise choose an odd modulus. Its gcd with each of the other three moduli must be 3, so all four moduli are divisible by 3. Every pair gcd is then 3. Disjointness requires four pairwise distinct residues modulo 3, again impossible. Therefore some pair has gcd at least 4.

The complete formal proof is [C2.lean](#).

1.4 C3 — Every size through eight

This cell is supplied as a complete formal proof, with all definitions and lemmas included in the repository. It is not justified by citing a published theorem. [C3.solution](#) has the exact type

```
∀ k : Nat, 2 ≤ k → k ≤ 8 → ProofPursuit.P4.Statement k
```

The dependency chain is explicit: size 2 follows from the CRT criterion; C1 and C2 prove sizes 3 and 4; [Five](#), [Six](#), and [Seven](#) prove sizes 5–7; [Eight.step](#) derives size 8 from size 7, and [Progress.solution](#) discharges that premise. [Parity.lean](#) contains the shared parity arguments. Every component uses actual congruence-class disjointness. No unproved custom axiom, sorry, or native evaluation axiom occurs.

The independent kernel check `lake env leanchecker ProofPursuit.P4.C3` passed. The only axioms are Lean’s standard `propext`, `Classical.choice`, and `Quot.sound`. All later smaller-case assumptions are supplied by these proved components, not left as assumptions in the final range theorem.

1.5 P4 C4: range proof and exact method audit

Task source: [Every k up to 12](#), checked on 2026-09-26. This cell is complete under the stated hand-in requirements. No organizer acceptance or platform submission is claimed.

1.5.1 Claim and meaning of the computation

For every integer k with $2 \leq k \leq 12$, every k pairwise disjoint integer congruence classes with positive moduli contain a pair whose moduli have gcd at least k . Repeated moduli are allowed.

A complete Lean proof is `ThroughTwelve.solution` in [ThroughTwelve.lean](#). Its theorem has no search-completion or smaller-case premises. The supplied `Statement` uses actual disjointness of integer sets. Bézout/CRT, normalization, the density inequality, enumeration correctness, and the concrete $k=11$ and $k=12$ refutations are proved inside the package. Lean 4.34.1 plus bundled `Std` suffices. Both a fresh build and a separate kernel replay passed; [verification record](#) gives commands and axiom audit.

For the requested audit at 9–16, distinguish two stages:

1. **Necessary modulus tests:** enumerate finite normalized modulus multisets, apply every-subset density and the explicitly described extra minimal-counterexample conditions.
2. **Actual residue decision:** decide every remaining multiset by complete finite residue search, with independently checked refutation trees and concrete witnesses proving minimum obstruction cardinalities.

The [reduction and pruning proof](#) supplies the mathematical justification. The [manifest](#) locates exact data and code. The residue tests concern all assignments, not a sample and not a bounded search over the original unbounded integers: the finite residue normalization is proved in that document.

1.5.2 Exact results

k	Raw density-only lists	Lists after extra modulus tests	Lists still undecided after residue certificates	
9	0	0		0
10	0	0		0
11	0	0		0
12	0	0		0
13	51	1		0
14	0	0		0
15	not certified in an unpruned run	30		0
16	not certified in an unpruned run	77		0

The chosen method at 15 and 16 includes extra necessary-condition pruning during enumeration. Its complete final survivor sets are the `reduced_survivors` arrays in [k=15](#) and [k=16](#). These files exhibit every modulus with multiplicity. For 9–14, the exact raw and reduced arrays are in [run-9-14.jsonl](#). A strong run’s intermediate `base_survivors` must not be described as the complete raw density-only set.

The second implementation chooses compatible distinct values and then their multiplicities, whereas the first grows occurrence lists from anchor triples. The recorded [elementwise comparison](#) has empty symmetric differences for every claimed final set. Fresh replay records node counts and wall clocks, and checks those counts against the baseline.

1.5.3 Why no surviving modulus list remains undecided

At $k=13$ the single reduced list is

`[10,10,10,10,10,12,12,12,12,24,36,40,45]`.

Its subset `[10,10,10,10,10,12,45]` is impossible. Disjointness from the 12-class forces all five 10-class residues to have the other parity. Pairwise disjointness among the 10-classes makes those five residues distinct modulo 10. They therefore exhaust all residues modulo 5. The 45-class meets one of them because its gcd with 10 is 5. This proves impossibility without computation. The 239 explicit assignments for every distinct nonempty submultiset of size at most six prove that seven is the smallest obstruction cardinality inside the full list. Both the impossibility and minimality are also Lean-certified in `Survivor13` and `Minimality13`.

At $k=15$ and 16, [global-minima-claims.jsonl](#) gives each of the 107 lists, its individual obstruction, and its minimum cardinality. Every obstruction has a complete refutation tree; there are 59 distinct trees and 40,292 nodes. Every smaller submultiset has an explicit residue witness, checked directly by the CRT gcd criterion. Thus no smaller submultiset can force the negative decision. Repeated values are treated as separate positions when defining subfamilies, and

duplicate submultisets share witnesses only because equal modulus multisets have exactly the same realizability question. The zero-entry subfamily is trivially realizable.

These facts also extend the mathematical conclusion inductively to $k=16$: assume a least failing size in 13–16, apply its justified finite reduction, and obtain a list which the exhaustive computation and residue certificates exclude. Every invocation of a smaller-size theorem is thereby discharged. This extension is computationally certified, not claimed to be formalized in Lean.

1.5.4 Source discrepancy, tested rather than assumed

O’Bryant’s [paper](#), §4.4, reports no survivors through 19 from the displayed pair-gcd and every-subset density machinery. The explicit thirteen-list above passes those tests: every pair gcd is in $[2,12]$, and all 8,178 position subsets of cardinality at least two satisfy the canonical integer density inequality. Independent Python checking and `DensityAudit13.every_sublist_passes` establish this. Consequently that reported emptiness claim, interpreted as the stated necessary-condition tests, is incorrect. Our exhibited survivor is not a counterexample to the disjoint-congruence conjecture; its residue impossibility was proved above. We do not claim to have identified the cause of the paper’s computational discrepancy.

During our own audit we found that the residue search used a stronger compressed-variable symmetry than the written explanation. We initially called this an error; further analysis shows the old variables were interchangeable because their effective bounds and gcd constraints agree. That initial diagnosis is withdrawn, with proof in the [reduction document](#). The current implementation uses the simpler equal-original-modulus rule and reproduces every stored claim and witness unchanged. Independent saved-tree replay has always used that simpler rule.

1.5.5 Fresh exhaustive replay

`python3 -B cagent/p4/audit/replay.py --max-k 16` completed in **500.013 seconds**. Both searches reproduced their exact saved node counts and survivor lists at all eight sizes; their relevant lists agree elementwise. All certificate checks passed. The [evidence file](#) records per-size wall clocks, node counts, completion flags, positive control, and certificate results. The runner uses a new output directory by default.

k	Tuple recursion nodes	Graph recursion nodes	Tuple wall seconds	Graph wall seconds
9	10	20	0.031	0.03
10	92	85	0.032	0.033
11	9,541	7,460	0.107	0.106
12	773	699	0.042	0.036
13	347,628	217,024	5.2	3.795
14	12,884	8,359	0.149	0.081
15	2,330,980	1,522,808	59.622	57.823
16	6,921,594	4,111,538	201.727	165.504

The two algorithms count different recursive states, so their counts need not equal one another. Each exactly matches its own earlier run. The underlying JSONL records also include depth-specific counts and anchor-triple attempts where applicable.

1.6 P4 C5: certified range and certificate checklist

Task source: [The certified boundary](#), checked on 2026-09-26. This package is not a platform submission or an organizer acceptance.

1.6.1 Claimed certified boundary: 14

C5 is solved for the contiguous claimed range 2–14, the largest range for which this package currently completes every C5 benchmark, including an observed rule-on/rule-off comparison. This endpoint describes our completed certificate, not the largest true size of the conjecture.

Sizes 2–12 have a complete Lean theorem. For 13, assume it is the least failing size, normalize a minimum-modulus-sum counterexample, and apply the [proved finite reduction and pruning rules](#). The complete modulus search leaves exactly the explicit thirteen-list; its seven-entry obstruction is impossible, as proved in the [C4 argument](#). Thus 13 holds. Repeating the least-counterexample argument at 14 gives an empty enumerated list, proving 14. Every smaller-size dependency is discharged in order.

The full computational replay for this C5 claim took **16.109 seconds**, including both independent implementations, exact node-count and survivor comparisons, all rule-on/off runs at 9–14, positive control, and the saved residue-certificate checks. Reproduce with:

```
python3 -B cagent/p4/audit/replay.py --max-k 14
```

It writes outputs to a new temporary directory by default, preserving bundled evidence. [Recorded replay](#) contains every size’s nodes, wall clock, exact baseline agreement, and equal final lists with early strong rules enabled and disabled. Counts agree exactly; elapsed time is machine-dependent.

Computational evidence also extends the mathematical argument through 16, but its additional C5 rule-off runs are unfinished. Those sizes are **not included in this C5 boundary claim**. No completed exhaustive search at 17 or above is claimed.

1.6.2 Isolated sizes

[Known-cases proof](#) reconstructs, line by line, the exclusions of **24 and 30 as least counterexample sizes**. It explicitly assumes all smaller sizes for each exclusion. It uses finite prime-support counting, not a search, and repairs an insufficient final inference in the source’s exceptional k=30 argument. These are not unconditional 24-class or 30-class theorems inferred from our range through 16.

1.6.3 Nonvacuous control

The unchanged base enumeration function accepts a gcd cap and a target length. Run `enumerate.py 7 --target-length 6` without the least-counterexample-only strong flag. It returns six copies of modulus 6, witnessed by residues `[0,1,2,3,4,5]`. These classes are pairwise disjoint; every pair gcd is 6, below the requested cap 7. The density is exactly one. The control uses the same finite domain, anchor enumeration, compatibility checks, density tests, and branching code as ordinary runs; it does not inject a preselected answer or disable a rejection because it happens to reject the desired example. The different cap and length are explicit inputs. Least-counterexample-only conditions cannot be imposed when those parameters differ, and the implementation rejects that invalid strong-mode combination.

The data are in [positive-control.jsonl](#), and fresh replay regenerates them. The control proves the machinery can return admissible objects; it is not a counterexample to the conjecture, whose cap and target size are equal.

1.6.4 Complete survivor decisions and smallest obstructions

Within the C5 claim, at 9–12 and 14 the final modulus survivor lists are empty; at 13 there is one reduced list. The additional, separately scoped work at 15 and 16 has 30 and 77 reduced lists. Every list is written out in full in the linked JSONL data, not represented only by a count. The [C4 solution](#) and [audit report](#) identify the files and the distinction between raw density and reduced lists.

The 13-list has a seven-entry obstruction, with concrete realizations of every smaller submultiset. Its impossibility and minimum cardinality are also kernel-checked in Lean. For each of the other 107 lists, `global-minima-claims.jsonl` selects an obstruction, `minimum-refutations.jsonl` excludes all its residue assignments, and `global-minima-facts.jsonl` supplies actual residues for every smaller submultiset. The replay verifies inclusion, cardinality, every CRT inequality, full coverage of smaller submultisets, and all branches of each obstruction tree. This proves minimum cardinality inside each individual survivor, not merely that deleting one position repairs a chosen obstruction.

1.6.5 Independent method and pruning comparison

The first enumeration grows nondecreasing occurrence lists after selecting an anchor triple with replacement. The second builds cliques of distinct compatible modulus values, then chooses positive multiplicities. Their divisor-generation routines also differ. Exact final lists are compared elementwise at every enumerated size. Different traversal node counts are expected; differences in final lists are not silently reconciled. The [reduction proof](#) explains every mathematical pruning rule and every capacity or early-future check.

All numerical rules are necessary conditions proved in that document; an unfinished run is never interpreted as emptiness. For every computational size in the C5 claim, 9–14, completed runs with all three early strong rules enabled and disabled have identical final reduced survivor lists. The recorded comparisons turn off early feasibility checks while retaining the same proved complete-tuple conditions: they compare the same mathematical output set. Sizes 2–8 use complete direct Lean proofs, so there is no computational pruning or survivor list to compare there.

1.7 C6 — Beyond the boundary

Not attempted. No claim about the general open conjecture or a new case beyond the stated certificates.

2 Appendix A — Complete finite reduction and pruning proofs

2.1 Proof of the finite-search rejection rules

This note proves the mathematical implications used by [enumerate.py](#), [clique.py](#), and the modulus and residue checks in [audit.py](#). The source is Kevin O’Bryant, *On Z.-W. Sun’s Disjoint Congruence Classes Conjecture*, [arXiv:math/0604347v2](#), especially Proposition 2 and Lemmas

5–6. The arguments are reconstructed below rather than delegated to that citation. They apply to a **least-size** counterexample, and the sum-minimal normalization applies after choosing, among counterexamples of that size, one minimizing the sum of the moduli. They do not by themselves prove a conjecture at a size whose smaller cases are unsettled. This document establishes pruning soundness; completeness of a particular finite run additionally requires its recorded **complete: true** result and replay of its output.

2.1.1 Starting hypothesis and finite normalization

Assume $k \geq 4$ is the least size of pairwise disjoint integer congruence classes $a_i \pmod{m_i}$, with positive moduli, for which every $\gcd(m_i, m_j) < k$. Choose such a system with minimal $\sum_i m_i$. By the two-class Chinese remainder theorem, classes $a_i \pmod{m_i}$ and $a_j \pmod{m_j}$ meet if and only if $\gcd(m_i, m_j) \mid a_i - a_j$. Thus all pair gcds are in $\{2, \dots, k-1\}$; a gcd of 1 cannot occur in a disjoint system. This is the paper’s [Proposition 2](#).

Let

$$N = \text{lcm}_{i < j} \gcd(m_i, m_j), \quad L_k = \text{lcm}(1, 2, \dots, k-1).$$

Certainly $N \mid L_k$. If a maximal prime-power factor p^e of some m_i did not divide N , every other m_j would have p -adic valuation below e : otherwise $p^e \mid \gcd(m_i, m_j) \mid N$. Dividing just m_i by p consequently leaves every $\gcd(m_i, m_j)$ unchanged, since its p -valuation was already less than e . All disjointness inequalities remain true by the Chinese remainder criterion, while the modulus sum decreases. This contradicts its choice. Every maximal prime-power factor of every m_i therefore divides N , so $m_i \mid N \mid L_k$. Also $N = \text{lcm}_i m_i$: every pair gcd divides its participating moduli, and every modulus divides N . This proves the finite reduction in [Lemma 6\(1\)](#), pp. 3–4.

Every prime divisor of any m_i is therefore a prime below k . A modulus 1 is impossible because all its pair gcds would be 1. No modulus is a prime power. To prove the latter, suppose $m_i = p^e$. Every modulus is divisible by p , since it has gcd greater than 1 with m_i ; also $p < k$. Among the k residues, choose $t = \lceil k/p \rceil$ with one common residue modulo p , where $2 \leq t < k$. Subtract that common residue and divide these selected residues and moduli by p . For every selected pair, both its residue difference and pair gcd are divided by p , hence the divided classes remain disjoint. Minimality of k forces a pair gcd at least t among them, making the original pair gcd at least $pt \geq k$, a contradiction. This proves [Lemma 6\(4\)](#), pp. 4–5. Thus a complete finite *candidate* domain for modulus **values** is the set of divisors of L_k having at least two distinct prime factors; later conditions may exclude individual values or combinations. Both `enumerate.domain` (prime-power product expansion) and `clique.divisors_with_two_primes` (divisor generation and trial division) compute that set; repetitions remain possible.

For every prime power $q \mid m_i$, also $q \mid N$. Since N is the lcm of pair gcds, some pair gcd is divisible by q , so at least two moduli are divisible by q . Whether or not i belongs to that pair, at least one of those two moduli is different from i . This proves the *prime-power support* rule, [Lemma 6\(3\)](#). The code tests the maximal power of each prime in each modulus; testing these powers suffices because every lower power is then supported as well. `enumerate.power_support`, `audit.maximal_power_failure`, and `clique.extra_good` all implement this implication on complete multisets.

2.1.2 Anchor counts

Call a modulus an anchor when $k-1 \mid m_i$. There must be at least three anchors. If there were at most two, delete one anchor, or any class if there are none. The remaining $k-1$ classes would

be disjoint and no pair gcd could equal $k - 1$, since that would require two remaining anchors. Every pair gcd would be below $k - 1$, contradicting the least-size choice. This is [Lemma 6\(5\)](#), and justifies beginning both searches with at least three anchors.

If there are **exactly** three anchors, there must be at least two *other* moduli divisible by $d = k - 2$. Here is a direct proof of [Lemma 6\(6\)](#). Retain any one anchor and delete the other two. The remaining $k - 2$ disjoint classes cannot have all pair gcds below $k - 2$; otherwise they form a smaller counterexample. Their gcds are still below k , and a gcd of $k - 1$ is impossible with only one anchor remaining. Hence some two remaining moduli are divisible by d . If no outside modulus is divisible by d , this is impossible. If exactly one outside modulus x is divisible by d , repeating the deletion with each of the three possible retained anchors shows that all three anchors are divisible by d . Two anchors would then have gcd divisible by $\text{lcm}(k - 1, k - 2) = (k - 1)(k - 2) \geq k$, impossible. Thus at least two outside moduli are d -divisible. `special_obstruction` and `extra_good` check exactly that outside count when the final anchor count is three.

2.1.3 The large-prime rule, including its exceptional case

For $7 \leq k \leq 30$, let $p \geq k/2$ be a prime dividing a modulus. Prime-power support gives at least two p -divisible moduli, and pair-gcd bounds imply $p < k$. Suppose there are $\ell \geq 3$ of them. For each such modulus m_i , let P_i contain its prime divisors other than p . Each P_i is nonempty by the prime-power exclusion. They are pairwise disjoint: a prime q in two of them would give a pair gcd divisible by $pq \geq 2p \geq k$. All these primes are below k by finite normalization.

Each of the other $k - \ell$ moduli meets every anchor in gcd greater than 1. It is not divisible by p , so it shares some prime in each P_i . Choosing one shared prime per anchor gives a signature in $P_1 \times \cdots \times P_\ell$. Equal signatures for two outside moduli would make their gcd divisible by the product of ℓ distinct primes. Because $\ell \geq 3$, this product is at least $2 \cdot 3 \cdot 5 = 30 \geq k$. The signature map is therefore injective, so

$$k - \ell \leq \prod_{i=1}^{\ell} |P_i|. \quad (1)$$

If r is the number of primes below k , then $\sum_i |P_i| \leq r - 1$: p is omitted and the P_i are disjoint. For a fixed positive-integer sum, the product is maximal when sizes differ by at most one; moving one unit from a to b when $a \geq b + 2$ raises the product by $a - b - 1$. Using all $r - 1$ available units gives the upper bounds below. A dash means $\ell > r - 1$, already impossible because each support is nonempty.

r	smallest possible k in $7 \leq k \leq 30$	$\ell = 3$	4	5	6	7	8
3	7	—	—	—	—	—	—
4	8	1	—	—	—	—	—
5	12	2	1	—	—	—	—
6	14	4	2	1	—	—	—
7	18	8	4	2	1	—	—
8	20	12	8	4	2	1	—
9	24	18	16	8	4	2	1
10	30	27	24	16	8	4	2

For rows $r = 3, \dots, 9$, each numeric bound is strictly below $k - \ell$, even using the row's smallest possible k . For $r = 10$, $k = 30$; every entry with $\ell \geq 4$ is below $30 - \ell$. The one equality is $k = 30, \ell = 3$, where (1) forces all three P_i to have size 3, to partition all nine primes other than p , and to realize all 27 signatures on the 27 outside moduli. Of the four primes 17, 19, 23, 29, precisely one can be p , so some $q \geq 17$ belongs to one support. Choose a prime q' in another support; $qq' \geq 34 > 30$. Fix the signature coordinates q, q' . The third coordinate has three choices, each realized by a distinct outside modulus. Any two of those moduli have gcd at least $qq' > 30$, a contradiction. Hence ℓ cannot exceed 2, and support forces $\ell = 2$. This is [Lemma 6\(8\)](#), pp. 5–6, with its final exceptional-case inference made explicit.

The paper's final sentence says two of 17, 19, 23, 29 must lie in separate P_i . That statement does not follow literally when the excluded prime p is one of the four: the other three could occupy one size-three support. The q, q' argument above covers this configuration. `enumerate.py` applies the rule only for $7 \leq k \leq 30$. `clique.py` currently tests $k \leq 30$ without the lower bound; its published runs begin at $k = 9$, so they are inside the proved range. Its generic strong-mode pruning for $k < 7$ is not justified by this argument.

2.1.4 Density for every position subset

For any subset S of at least two *positions*, define $M_S = \text{lcm}_{i < j \in S} \text{gcd}(m_i, m_j)$. Modulo M_S , the class $a_i \pmod{m_i}$ occupies exactly $M_S / \text{gcd}(m_i, M_S)$ residue classes. These occupied sets for distinct $i \in S$ must be disjoint: a common residue modulo M_S would make $a_i - a_j$ divisible by $\text{gcd}(m_i, m_j)$, since that gcd divides M_S , and the original two classes would meet. Counting the M_S available residues gives

$$\sum_{i \in S} \frac{M_S}{\text{gcd}(m_i, M_S)} \leq M_S. \quad (2)$$

Thus an exact integer total greater than M_S rejects the tuple, as in [Lemma 5](#) and [Lemma 6\(7\)](#), pp. 2–3. If M is any multiple of M_S , then $\text{gcd}(m_i, M_S) \mid \text{gcd}(m_i, M)$, so each reciprocal term for M is no larger than for M_S . Testing M_S therefore covers every larger permitted period. For a pair, (2) follows automatically from pair gcd at least 2: its two terms total $2/M_S \leq 1$. It suffices for the scripts to enumerate subset sizes at least three. Repeated values still represent different positions, so their multiplicities are retained in the sum.

`density_obstruction`, `density_bad`, and `direct_density_witness` compute (2) with integer `lcm`, `gcd`, and division. A bad prefix, triple, or other selected position subset stays in every extension, making each early density rejection hereditary. In `clique.py`, once c copies of a value form a bad prefix or triple, increasing c cannot repair that already-present subset; the loop's `break` is sound. At a leaf, `first_bad_subset` and `all_subsets_good` enumerate every position subset of size at least three. In particular, rejection does not depend on a floating-point threshold or on a density heuristic.

2.1.5 Search-path and partial-pool coverage

`enumerate.py` constructs every nondecreasing anchor triple with replacement. For any eligible k -multiset, its first three anchors give exactly one such triple; the remaining occurrences can then be appended in nondecreasing order from `pool`. The initial pool contains every nonanchor compatible with the triple and every compatible anchor no smaller than its third value, **including equal values**. At each extension, `following = pool[i:]` preserves exactly the allowed nondecreasing suffix, intersected with compatibility with the newly chosen value. Thus reject-

ing $\text{gcd} \leq 1$ or $\text{gcd} \geq k$ at the triple or extension level removes only impossible pairs, and every eligible multiset has a search path. The `--target-length` option is used for diagnostics; strong minimal-counterexample pruning is explicitly forbidden by the code when `target_length != k`.

`clique.py` independently enumerates distinct compatible modulus values, gives each a positive multiplicity, and sorts the result. A repeated value $x \geq k$ is impossible because its pair gcd would be $x \geq k$. A repeated value $x < k$ has multiplicity at most x : for c identical classes, (2) with $M_S = x$ says $c/x \leq 1$. These are the `max_count` bounds. Candidate distinct values form a clique under pair compatibility. Selecting all anchor values first and then all nonanchor values loses no multiset, since order of modulus positions has no mathematical effect. At the anchor stage, `possible_others` contains every nonanchor compatible with the chosen anchors; at the other stage, each recursive `allowed` suffix contains every later distinct compatible value. The capacity rejection

`len(xs) + sum(min(k,x) if x < k else 1 for x in candidates) < k`

uses an **upper** bound on how many positions the remaining values can supply. If even that bound cannot reach k , no completion exists. The analogous anchor-stage inequality with right side 3 only rejects states unable to reach the required three anchors. `expand_others` is called at each anchor state of length at least three, so all possible final anchor counts are covered. Its and `expand_anchors`'s density and triple breaks have the hereditary justification above.

Both strong modes use an overapproximation of future choices, which makes their three no-future rejections safe:

1. **Prime-power support.** If a maximal power q now appears once and no value in the future pool is divisible by q , it can never attain the required second occurrence.
2. **Large-prime count.** For $7 \leq k \leq 30$, more than two current multiples of $p \geq k/2$ cannot be removed. Exactly one current multiple with no future p -multiple cannot reach the required two. (`clique.py` has the lower-bound caveat noted above.)
3. **Exactly three anchors.** If there are exactly three current anchors, no future anchor, fewer than two current **outside** $k - 2$ -multiples, and no future $k - 2$ -multiple, the final tuple violates the proved exact-three rule.

In `enumerate.py`, `pool` is the complete sorted suffix of compatible future values. In `clique.py`, `candidates` is the complete suffix of distinct future values in `expand_others`; at an anchor state, `candidates + possible_others` is a possibly larger-than-real set of future values. Missing support from these pools therefore implies missing support from every actual continuation. The strong partial rules need no estimate of remaining positions; overlooking an impossible branch is harmless, while every branch they reject violates a necessary condition. At complete tuples, `special_obstruction` and `extra_good` apply the same three conditions directly.

2.1.6 Residue-checking caveat and audit scope

For a fixed modulus tuple, pairwise disjointness depends only on residues modulo $R_i = \text{lcm}_{j \neq i} \text{gcd}(m_i, m_j)$, and $R_i \mid m_i$. Common translation can set the first residue to zero; equal original moduli can be permuted so their residues increase with index. These are sound finite-domain reductions for an exact residue search. A branch is impossible if a proposed residue has pair-gcd divisibility with an assigned one, if an equal-original-modulus order is violated, or if an unassigned variable has no admissible residue. The separately exported trees in `export_refutations.py` and independently replayed trees in `verify_refutations.py` use the

equal-original-modulus condition $xs[j] == xs[i]$ and check every admissible child and each dead leaf.

An earlier `audit.py` version compared $xs[j]$ to the effective bound R_i , rather than to $xs[i]$. Our first audit called this unsound because the original moduli could differ. **That diagnosis was too strong and is withdrawn.** The old comparison also describes a valid compressed symmetry: if $m_j = R_i = q$, then $q \mid m_i$ and every $\gcd(m_i, m_\ell)$ divides q . For every other position ℓ , therefore,

$$\gcd(m_i, m_\ell) = \gcd(q, m_\ell) = \gcd(m_j, m_\ell).$$

Moreover $\gcd(m_i, m_j) = q$ and $R_j = q$. The two finite residue variables have identical bounds and identical constraints with all other variables. They may be exchanged even if their original moduli differ. Within each such effective-modulus group, residues are distinct modulo q and can be sorted by index; this satisfies all comparisons imposed by the old code. Thus we found a mismatch between the documented symmetry and its implementation, not a counterexample to search soundness.

The current code uses the simpler equal-original-modulus comparison, matching the proof above and the independent tree verifier. A fresh run from empty output files reproduced all 107 claims, all 20,436 fact keys and statuses, and every explicit SAT witness unchanged. The minimum verifier checked all 19,704 required smaller SAT facts; the tree verifier replayed all 59 refutations and 40,292 nodes. Both versions have sound symmetry reductions; the package relies on the current version and these independent certificates. Modulus enumeration never calls this residue solver.

3 Appendix B — Isolated least-counterexample exclusions

3.1 Conditional exclusions of least counterexamples at 24 and 30

Kevin O’Bryant’s *On Z.-W. Sun’s Disjoint Congruence Classes Conjecture*, [arXiv:math/0604347v2](https://arxiv.org/abs/math/0604347v2), Theorem 3 and Lemma 6, states that a **least-size** counterexample cannot have size 24 or 30. The following reconstructs the needed part of Lemma 6, including its finite prime-count argument. It does **not** prove the conjecture independently at either size: the argument assumes the conjecture at every smaller size. No computer search or Lean proof is claimed here.

Suppose, for contradiction, that $k \in \{24, 30\}$ is the least number of pairwise disjoint integer congruence classes $a_i \pmod{m_i}$ with positive moduli and

$$\gcd(m_i, m_j) < k \quad (i \neq j).$$

Among counterexamples of this size, choose one minimizing $\sum_i m_i$. The Chinese remainder criterion says that two classes are disjoint exactly when their pair gcd does **not** divide their residue difference (Proposition 2, p. 2). Consequently every pair gcd is greater than 1: a gcd of 1 would make those two classes meet.

3.1.1 Normalization and prime support

Put $N = \text{lcm}_{i < j} \gcd(m_i, m_j)$ and $L_k = \text{lcm}(1, \dots, k-1)$. Since every pair gcd is a positive integer below k , $N \mid L_k$. Moreover, each $m_i \mid N$. To see the latter without assuming it, take a maximal

prime power $p^e \mid m_i$. If $p^e \nmid N$, every other m_j has p -adic valuation below e . Replacing m_i by m_i/p therefore preserves **every** pair gcd involving i , hence preserves all disjointness conditions by the Chinese remainder criterion. It also lowers $\sum_i m_i$, a contradiction. Applying this to each prime-power factor of m_i proves $m_i \mid N$. Thus every prime dividing any modulus is below k . In fact $N = \text{lcm}_i m_i$, because each pair gcd divides each participating modulus, while every modulus divides N . This is [Lemma 6\(1\)](#), pp. 3–4.

Any prime power $q \mid m_i$ divides N , so by the definition of an lcm it divides at least one pair gcd. Hence it divides at least two moduli; in particular, another modulus besides m_i is divisible by q . This is [Lemma 6\(3\)](#). The moduli cannot be prime powers. Indeed, if $m_i = p^e$, every m_j is divisible by p , since all $\text{gcd}(m_i, m_j) > 1$. Also $p < k$. At least $t = \lceil k/p \rceil$ of the residues have one common value modulo p ; select exactly t such classes. Here $2 \leq t < k$. Subtract their common residue and divide the selected residues and moduli by p . The resulting t classes are pairwise disjoint: for two selected indices, both the residue difference and the pair gcd are divided by p , so nondivisibility is preserved. The conjecture at size $t < k$, available from the least-size assumption, forces a divided pair gcd at least t . Its original pair gcd is then at least $pt \geq k$, a contradiction. This reconstructs [Lemma 6\(4\)](#), pp. 4–5. A modulus 1 is already excluded by the pair-gcd bound, so every modulus divisible by a prime p has another distinct prime factor.

Now let $p = k - 1$, which is respectively 23 or 29 and is prime. At least three moduli are divisible by p . Otherwise delete one of the at most two p -divisible classes (or any class if there are none). The remaining $k - 1$ classes are disjoint. Their pair gcds are below k , and none can equal $k - 1 = p$, because no two remaining moduli are divisible by p . They would be a smaller counterexample. This is the relevant instance of [Lemma 6\(5\)](#), p. 5.

3.1.2 Counting the possible supports

Write $m_1, \dots, m_\ell$ for **all** the moduli divisible by p , so $\ell \geq 3$, and let P_i be the set of prime divisors of m_i other than p , for $1 \leq i \leq \ell$. Every P_i is nonempty by the prime-power exclusion above. These P_i are pairwise disjoint: if distinct anchors both contained $q \neq p$, their gcd would be divisible by $pq \geq 2(k - 1) > k$. Every prime in every P_i is below k , by $m_i \mid N \mid L_k$.

For each outside modulus m_j , $j > \ell$, and each anchor $i \leq \ell$, the pair gcd is greater than 1 but $p \nmid m_j$. Thus m_i and m_j share some prime in P_i . Choose one such prime $q_i(j)$, for instance the smallest, and assign m_j the signature $(q_1(j), \dots, q_\ell(j)) \in P_1 \times \dots \times P_\ell$. If two outside indices had the same signature, their gcd would be divisible by the product of those ℓ **distinct** primes. As $\ell \geq 3$, that product is at least $2 \cdot 3 \cdot 5 = 30 \geq k$. Hence the signature map is injective, and

$$k - \ell \leq \prod_{i=1}^{\ell} |P_i|. \quad (1)$$

These are the support and injection steps in the proof of [Lemma 6\(8\)](#), pp. 5–6. There are nine primes below 24 and ten below 30. Excluding p , the disjoint nonempty supports have total size at most 8 or 9 respectively. For positive integers $s_i = |P_i|$ of fixed sum, the product is maximized when the sizes differ by at most one: replacing (a, b) with $(a - 1, b + 1)$ when $a \geq b + 2$ increases the product by $a - b - 1 > 0$. Using the entire available prime budget can only increase the maximum. The resulting bounds are:

ℓ	maximum $\prod s_i$, $k = 24$	$24 - \ell$	maximum $\prod s_i$, $k = 30$	$30 - \ell$
3	18	21	27	27
4	16	20	24	26

ℓ	maximum $\prod s_i$, $k = 24$	$24 - \ell$	maximum $\prod s_i$, $k = 30$	$30 - \ell$
5	8	19	16	25
6	4	18	8	24
7	2	17	4	23
8	1	16	2	22
9	impossible	—	1	21

For $k = 24$, every row contradicts (1). For $k = 30$, every row except $\ell = 3$ contradicts (1). In that exceptional row, equality throughout is forced: each of the three supports has size 3, together they partition all nine primes 2, 3, 5, 7, 11, 13, 17, 19, 23, and the 27 outside indices realize **every** one of the $3^3 = 27$ signatures.

The prime 17 lies in one support; choose any prime q in a different support. Then $17q \geq 34 > 30$. Fix 17 and q in their two signature coordinates. The third coordinate has three choices, all realized by distinct outside moduli. Any two of those moduli both contain 17 and q , so their gcd is at least $17q > 30$, the final contradiction.

The paper’s last sentence on this exceptional case says that two of 17, 19, 23, 29 must lie in separate P_i . Taken literally for $p = 29$, this does not follow: p is excluded from every P_i , and 17, 19, 23 could occupy the same three-element support. The preceding 17-and- q argument covers that arrangement and supplies the needed conclusion. The paper’s theorem statement is thus supported here by a corrected final counting step, rather than by that particular sentence.

4 Appendix C — Complete formal proof of C3

For completeness, the entire local dependency closure of `C3.solution` follows in dependency order. Each block is the exact verified source file named above it. Only bundled Lean/Std imports are external. The repository keeps these as separate files for replay; the combined write-up includes every proof term and tactic step.

4.0.1 ProofPursuit/P4/Basic.lean

```
import Std

namespace ProofPursuit.P4

/-- Membership in the integer congruence class of a modulo the natural modulus m. -/
def InClass (a : Int) (m : Nat) (z : Int) : Prop := (m : Int) ∣ z - a

/-- The actual set-theoretic disjointness condition, not a CRT assumption. -/
def DisjointClasses {k : Nat} (a : Fin k → Int) (m : Fin k → Nat) : Prop :=
  ∀ i j, i ≠ j → ¬ ∃ z : Int, InClass (a i) (m i) z ∧ InClass (a j) (m j) z

def Statement (k : Nat) : Prop :=
  ∀ (a : Fin k → Int) (m : Fin k → Nat),
    (∀ i, 0 < m i) → DisjointClasses a m →
      ∃ i j : Fin k, i < j ∧ k ≤ Nat.gcd (m i) (m j)

/-- Extended Euclid, proved from the standard library's gcd induction. -/
theorem bezout (m n : Nat) :
  ∃ x y : Int, (Nat.gcd m n : Int) = (m : Int) * x + (n : Int) * y := by
  induction m, n using Nat.gcd.induction with
  | H0 n => exact ⟨0, 1, by simp⟩
```

```

| H1 m n _ ih =>
  obtain ⟨x, y, h⟩ := ih
  refine ⟨y - (n / m : Nat) * x, x, ?_⟩
  rw [Nat.gcd_rec]
  have he : (n % m : Nat) + (m : Int) * (n / m : Nat) = (n : Int) := by
    exact_mod_cast Nat.mod_add_div n m
  grind

/-- The intersection direction of the generalized Chinese remainder theorem. -/
theorem meet_of_gcd_dvd (a b : Int) (m n : Nat)
  (h : (Nat.gcd m n : Int) | b - a) :
  ∃ z : Int, InClass a m z ∧ InClass b n z := by
  obtain ⟨x, y, hb⟩ := bezout m n
  obtain ⟨t, ht⟩ := h
  refine ⟨a + (m : Int) * x * t, ?_, ?_⟩
  · exact ⟨x * t, by grind⟩
  · exact ⟨-y * t, by grind⟩

theorem incompatible {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (h : DisjointClasses a m) {i j : Fin k} (hij : i ≠ j) :
  ¬ (Nat.gcd (m i) (m j) : Int) | a j - a i := by
  intro hd
  exact h i j hij (meet_of_gcd_dvd _ _ _ hd)

end ProofPursuit.P4

namespace ProofPursuit.P4

theorem bounds {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (hm : ∀ i, 0 < m i) (hd : DisjointClasses a m)
  (hn : ¬ ∃ i j : Fin k, i < j ∧ k ≤ Nat.gcd (m i) (m j))
  {i j : Fin k} (hij : i ≠ j) :
  2 ≤ Nat.gcd (m i) (m j) ∧ Nat.gcd (m i) (m j) < k := by
  have hp := Nat.gcd_pos_of_pos_left (m j) (hm i)
  have hne : Nat.gcd (m i) (m j) ≠ 1 := by
    intro h
    have hi := incompatible hd hij
    rw [h] at hi
    exact hi (by simp)
  have hu : Nat.gcd (m i) (m j) < k := by
    apply Classical.byContradiction
    intro h
    have hl : k ≤ Nat.gcd (m i) (m j) := by omega
    by_cases hlt : i < j
    · exact hn ⟨i, j, hlt, hl⟩
    · exact hn ⟨j, i, by omega, by simp [Nat.gcd_comm] using hl⟩
  omega

theorem residues_ne {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (hd : DisjointClasses a m) {i j : Fin k} (hij : i ≠ j) {d : Nat}
  (hg : Nat.gcd (m i) (m j) = d) : a i % (d : Int) ≠ a j % (d : Int) := by
  intro h
  apply incompatible hd hij
  rw [hg, Int.dvd_iff_emod_eq_zero]
  exact Int.emod_eq_emod_iff_emod_sub_eq_zero.mp h.symm

end ProofPursuit.P4

namespace ProofPursuit.P4

theorem residues_ne_of_gcd_dvd {k : Nat} {a : Fin k → Int} {m : Fin k → Nat}
  (hd : DisjointClasses a m) {i j : Fin k} (hij : i ≠ j) {d : Nat}
  (hg : Nat.gcd (m i) (m j) | d) : a i % (d : Int) ≠ a j % (d : Int) := by

```

```

intro h
apply incompatible hd hij
have hg' : (Nat.gcd (m i) (m j) : Int) | (d : Int) := by exact_mod_cast hg
exact Int.dvd_trans hg' (Int.dvd_iff_emod_eq_zero.mpr
  (Int.emod_eq_emod_iff_emod_sub_eq_zero.mp h.symm))

end ProofPursuit.P4

```

4.0.2 ProofPursuit/P4/C1.lean

```

import ProofPursuit.P4.Basic

namespace ProofPursuit.P4.C1

def target : Prop := Statement 3

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hg (i j : Fin 3) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 2 := by
    have := bounds hm hd hn hij
    omega
  have h01 := residues_ne hd (by decide : (0 : Fin 3) ≠ 1) (hg 0 1 (by decide))
  have h02 := residues_ne hd (by decide : (0 : Fin 3) ≠ 2) (hg 0 2 (by decide))
  have h12 := residues_ne hd (by decide : (1 : Fin 3) ≠ 2) (hg 1 2 (by decide))
  omega

end ProofPursuit.P4.C1

```

4.0.3 ProofPursuit/P4/C2.lean

```

import ProofPursuit.P4.Basic

namespace ProofPursuit.P4.C2

def target : Prop := Statement 4

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 4} (hij : i ≠ j) := bounds hm hd hn hij
  by_cases he : ∀ i, 2 ∣ m i
  • have hg (i j : Fin 4) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 2 := by
    have h := hb hij
    have hdiv := Nat.dvd_gcd (he i) (he j)
    have hz := Nat.mod_eq_zero_of_dvd hdiv
    omega
  have h01 := residues_ne hd (by decide : (0 : Fin 4) ≠ 1) (hg 0 1 (by decide))
  have h02 := residues_ne hd (by decide : (0 : Fin 4) ≠ 2) (hg 0 2 (by decide))
  have h12 := residues_ne hd (by decide : (1 : Fin 4) ≠ 2) (hg 1 2 (by decide))
  omega
  • obtain ⟨i, hi⟩ := Classical.not_forall.mp he
  have hg (j : Fin 4) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
    have h := hb hij
    have htwo : Nat.gcd (m i) (m j) ≠ 2 := by
      intro hh
      apply hi
    have hx := Nat.gcd_dvd_left (m i) (m j)
    rwa [hh] at hx

```

```

    omega
have hthre : ∀ j, 3 ∣ m j := by
  intro j
  by_cases hij : i = j
  · subst j
    have hj : ∃ j : Fin 4, i ≠ j := by
      by_cases h : i = 0
      · exact ⟨1, by omega⟩
      · exact ⟨0, h⟩
    obtain ⟨j, hj⟩ := hj
    have hx := Nat.gcd_dvd_left (m i) (m j)
    rwa [hg j hj] at hx
  · have hx := Nat.gcd_dvd_right (m i) (m j)
    rwa [hg j hij] at hx
have hall (i j : Fin 4) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
  have h := hb hij
  have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hthre i) (hthre j))
  omega
have h01 := residues_ne hd (by decide : (0 : Fin 4) ≠ 1) (hall 0 1 (by decide))
have h02 := residues_ne hd (by decide : (0 : Fin 4) ≠ 2) (hall 0 2 (by decide))
have h03 := residues_ne hd (by decide : (0 : Fin 4) ≠ 3) (hall 0 3 (by decide))
have h12 := residues_ne hd (by decide : (1 : Fin 4) ≠ 2) (hall 1 2 (by decide))
have h13 := residues_ne hd (by decide : (1 : Fin 4) ≠ 3) (hall 1 3 (by decide))
have h23 := residues_ne hd (by decide : (2 : Fin 4) ≠ 3) (hall 2 3 (by decide))
omega
end ProofPursuit.P4.C2

```

4.0.4 ProofPursuit/P4/Five.lean

```

import ProofPursuit.P4.Basic

namespace ProofPursuit.P4.Five

def target : Prop := Statement 5

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 5} (hij : i ≠ j) := bounds hm hd hn hij
  by_cases he : ∀ i, 2 ∣ m i
  · have hg (i j : Fin 5) (hij : i ≠ j) : Nat.gcd (m i) (m j) ∣ 4 := by
      have h := hb hij
      have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (he i) (he j))
      have hh : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 4 := by omega
      rcases hh with hh | hh <|> simp [hh]
      have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 1) (hg 0 1 (by decide))
      have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 2) (hg 0 2 (by decide))
      have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 3) (hg 0 3 (by decide))
      have h04 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 5) ≠ 4) (hg 0 4 (by decide))
      have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 5) ≠ 2) (hg 1 2 (by decide))
      have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 5) ≠ 3) (hg 1 3 (by decide))
      have h14 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 5) ≠ 4) (hg 1 4 (by decide))
      have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 5) ≠ 3) (hg 2 3 (by decide))
      have h24 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 5) ≠ 4) (hg 2 4 (by decide))
      have h34 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 5) ≠ 4) (hg 3 4 (by decide))
      omega
  · obtain ⟨i, hi⟩ := Classical.not_forall.mp he
  have hg (j : Fin 5) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
    have h := hb hij
    have hn2 : ¬ 2 ∣ Nat.gcd (m i) (m j) := by

```

```

    intro hh
    exact hi (Nat.dvd_trans hh (Nat.gcd_dvd_left _ _))
    have hz : Nat.gcd (m i) (m j) % 2 ≠ 0 := by
      intro hh
      exact hn2 (Nat.dvd_of_mod_eq_zero hh)
    omega
    have hthree : ∀ j, 3 ∣ m j := by
      intro j
      by_cases hij : i = j
      · subst j
        have hj : ∃ j : Fin 5, i ≠ j := by
          by_cases h : i = 0
          · exact ⟨1, by omega⟩
          · exact ⟨0, h⟩
        obtain ⟨j, hj⟩ := hj
        have hx := Nat.gcd_dvd_left (m i) (m j)
        rwa [hg j hj] at hx
      · have hx := Nat.gcd_dvd_right (m i) (m j)
        rwa [hg j hij] at hx
    have hall (i j : Fin 5) (hij : i ≠ j) : Nat.gcd (m i) (m j) = 3 := by
      have h := hb hij
      have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hthree i) (hthree j))
      omega
    have h01 := residues_ne hd (by decide : (0 : Fin 5) ≠ 1) (hall 0 1 (by decide))
    have h02 := residues_ne hd (by decide : (0 : Fin 5) ≠ 2) (hall 0 2 (by decide))
    have h03 := residues_ne hd (by decide : (0 : Fin 5) ≠ 3) (hall 0 3 (by decide))
    have h12 := residues_ne hd (by decide : (1 : Fin 5) ≠ 2) (hall 1 2 (by decide))
    have h13 := residues_ne hd (by decide : (1 : Fin 5) ≠ 3) (hall 1 3 (by decide))
    have h23 := residues_ne hd (by decide : (2 : Fin 5) ≠ 3) (hall 2 3 (by decide))
    omega

```

end ProofPursuit.P4.Five

4.0.5 ProofPursuit/P4/Six.lean

```

import ProofPursuit.P4.Basic

set_option maxHeartbeats 4000000

namespace ProofPursuit.P4.Six

def target : Prop := Statement 6

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 6} (hij : i ≠ j) := bounds hm hd hn hij
  have ncommon (p : Nat) (hp : p = 2 ∨ p = 3 ∨ p = 5) : ¬ ∀ i, p ∣ m i := by
    intro hc
    have hg (i j : Fin 6) (hij : i ≠ j) := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hc i) (hc j))
    rcases hp with rfl | rfl | rfl
    · have hh (i j : Fin 6) (hij : i ≠ j) : Nat.gcd (m i) (m j) ∣ 4 := by
        have h := hb hij
        have hz := hg i j hij
        have hx : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 4 := by omega
        rcases hx with hx | hx <|> simp [hx]
    have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 1) (hh 0 1 (by decide))
    have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 2) (hh 0 2 (by decide))
    have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 3) (hh 0 3 (by decide))
    have h04 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 4) (hh 0 4 (by decide))
    have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 2) (hh 1 2 (by decide))

```

```

have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 3) (hh 1 3 (by decide))
have h14 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 4) (hh 1 4 (by decide))
have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 3) (hh 2 3 (by decide))
have h24 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 4) (hh 2 4 (by decide))
have h34 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 6) ≠ 4) (hh 3 4 (by decide))
omega
• have hh (i j : Fin 6) (hij : i ≠ j) : Nat.gcd (m i) (m j) | 3 := by
  have h := hb hij
  have hz := hg i j hij
  have hx : Nat.gcd (m i) (m j) = 3 := by omega
  simp [hx]
  have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 1) (hh 0 1 (by decide))
  have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 2) (hh 0 2 (by decide))
  have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 3) (hh 0 3 (by decide))
  have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 2) (hh 1 2 (by decide))
  have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 3) (hh 1 3 (by decide))
  have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 3) (hh 2 3 (by decide))
  omega
• have hh (i j : Fin 6) (hij : i ≠ j) : Nat.gcd (m i) (m j) | 5 := by
  have h := hb hij
  have hz := hg i j hij
  have hx : Nat.gcd (m i) (m j) = 5 := by omega
  simp [hx]
  have h01 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 1) (hh 0 1 (by decide))
  have h02 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 2) (hh 0 2 (by decide))
  have h03 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 3) (hh 0 3 (by decide))
  have h04 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 4) (hh 0 4 (by decide))
  have h05 := residues_ne_of_gcd_dvd hd (by decide : (0 : Fin 6) ≠ 5) (hh 0 5 (by decide))
  have h12 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 2) (hh 1 2 (by decide))
  have h13 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 3) (hh 1 3 (by decide))
  have h14 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 4) (hh 1 4 (by decide))
  have h15 := residues_ne_of_gcd_dvd hd (by decide : (1 : Fin 6) ≠ 5) (hh 1 5 (by decide))
  have h23 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 3) (hh 2 3 (by decide))
  have h24 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 4) (hh 2 4 (by decide))
  have h25 := residues_ne_of_gcd_dvd hd (by decide : (2 : Fin 6) ≠ 5) (hh 2 5 (by decide))
  have h34 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 6) ≠ 4) (hh 3 4 (by decide))
  have h35 := residues_ne_of_gcd_dvd hd (by decide : (3 : Fin 6) ≠ 5) (hh 3 5 (by decide))
  have h45 := residues_ne_of_gcd_dvd hd (by decide : (4 : Fin 6) ≠ 5) (hh 4 5 (by decide))
  omega
have supports (i j : Fin 6) (hij : i ≠ j) :
  (2 | m i ∧ 2 | m j) ∨ (3 | m i ∧ 3 | m j) ∨ (5 | m i ∧ 5 | m j) := by
  have h := hb hij
  have hl := Nat.gcd_dvd_left (m i) (m j)
  have hr := Nat.gcd_dvd_right (m i) (m j)
  have hx : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 3 ∨
    Nat.gcd (m i) (m j) = 4 ∨ Nat.gcd (m i) (m j) = 5 := by omega
  rcases hx with hx | hx | hx | hx
  • exact Or.inl ⟨by simp [hx] using hl, by simp [hx] using hr⟩
  • exact Or.inr (Or.inl ⟨by simp [hx] using hl, by simp [hx] using hr⟩)
  • exact Or.inl ⟨Nat.dvd_trans (by decide : 2 | 4) (by simp [hx] using hl),
    Nat.dvd_trans (by decide : 2 | 4) (by simp [hx] using hr)⟩
  • exact Or.inr (Or.inr ⟨by simp [hx] using hl, by simp [hx] using hr⟩)
have two (i : Fin 6) : (2 | m i ∧ 3 | m i) ∨ (2 | m i ∧ 5 | m i) ∨
  (3 | m i ∧ 5 | m i) := by
  have nc2 := ncommon 2 (by simp)
  have nc3 := ncommon 3 (by simp)
  have nc5 := ncommon 5 (by simp)
  by_cases h2 : 2 | m i <=> by_cases h3 : 3 | m i <=> by_cases h5 : 5 | m i
  all_goals first
    | exact Or.inl ⟨h2, h3⟩
    | exact Or.inr (Or.inl ⟨h2, h5⟩)
    | exact Or.inr (Or.inr ⟨h3, h5⟩)
    | (ex falso; apply nc2; intro j; by_cases hij : i = j

```

```

    • simp [← hij] using h2
    • rcases supports i j hij with h | h | h
      • exact h.2
      • exact False.elim (h3 h.1)
      • exact False.elim (h5 h.1))
  | (ex falso; apply nc3; intro j; by_cases hij : i = j
    • simp [← hij] using h3
    • rcases supports i j hij with h | h | h
      • exact False.elim (h2 h.1)
      • exact h.2
      • exact False.elim (h5 h.1))
  | (ex falso; apply nc5; intro j; by_cases hij : i = j
    • simp [← hij] using h5
    • rcases supports i j hij with h | h | h
      • exact False.elim (h2 h.1)
      • exact False.elim (h3 h.1)
      • exact h.2)
  | (have hx : ∃ j : Fin 6, i ≠ j := by
    by_cases h : i = 0
    • exact ⟨1, by omega⟩
    • exact ⟨0, h⟩
    obtain ⟨j, hj⟩ := hx
    rcases supports i j hj with h | h | h
    • exact False.elim (h2 h.1)
    • exact False.elim (h3 h.1)
    • exact False.elim (h5 h.1))
have clash (i j : Fin 6) (hij : i ≠ j)
  (h : (2 | m i ∧ 3 | m i ∧ 2 | m j ∧ 3 | m j) ∨
    (2 | m i ∧ 5 | m i ∧ 2 | m j ∧ 5 | m j) ∨
    (3 | m i ∧ 5 | m i ∧ 3 | m j ∧ 5 | m j)) : False := by
  have hb' := hb hij
  rcases h with h | h | h
  all_goals
    have hx := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 h.2.2.1)
    have hy := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.2.1 h.2.2.2)
    omega
have h0 := two 0
have h1 := two 1
have h2 := two 2
have h3 := two 3
rcases h0 with h0 | h0 | h0 <;> rcases h1 with h1 | h1 | h1 <;>
  rcases h2 with h2 | h2 | h2 <;> rcases h3 with h3 | h3 | h3
all_goals first
  | exact clash 0 1 (by decide) ((Or.inl) ⟨h0.1, h0.2, h1.1, h1.2⟩)
  | exact clash 0 1 (by decide) ((Or.inr ∘ Or.inl) ⟨h0.1, h0.2, h1.1, h1.2⟩)
  | exact clash 0 1 (by decide) ((Or.inr ∘ Or.inr) ⟨h0.1, h0.2, h1.1, h1.2⟩)
  | exact clash 0 2 (by decide) ((Or.inl) ⟨h0.1, h0.2, h2.1, h2.2⟩)
  | exact clash 0 2 (by decide) ((Or.inr ∘ Or.inl) ⟨h0.1, h0.2, h2.1, h2.2⟩)
  | exact clash 0 2 (by decide) ((Or.inr ∘ Or.inr) ⟨h0.1, h0.2, h2.1, h2.2⟩)
  | exact clash 0 3 (by decide) ((Or.inl) ⟨h0.1, h0.2, h3.1, h3.2⟩)
  | exact clash 0 3 (by decide) ((Or.inr ∘ Or.inl) ⟨h0.1, h0.2, h3.1, h3.2⟩)
  | exact clash 0 3 (by decide) ((Or.inr ∘ Or.inr) ⟨h0.1, h0.2, h3.1, h3.2⟩)
  | exact clash 1 2 (by decide) ((Or.inl) ⟨h1.1, h1.2, h2.1, h2.2⟩)
  | exact clash 1 2 (by decide) ((Or.inr ∘ Or.inl) ⟨h1.1, h1.2, h2.1, h2.2⟩)
  | exact clash 1 2 (by decide) ((Or.inr ∘ Or.inr) ⟨h1.1, h1.2, h2.1, h2.2⟩)
  | exact clash 1 3 (by decide) ((Or.inl) ⟨h1.1, h1.2, h3.1, h3.2⟩)
  | exact clash 1 3 (by decide) ((Or.inr ∘ Or.inl) ⟨h1.1, h1.2, h3.1, h3.2⟩)
  | exact clash 1 3 (by decide) ((Or.inr ∘ Or.inr) ⟨h1.1, h1.2, h3.1, h3.2⟩)
  | exact clash 2 3 (by decide) ((Or.inl) ⟨h2.1, h2.2, h3.1, h3.2⟩)
  | exact clash 2 3 (by decide) ((Or.inr ∘ Or.inl) ⟨h2.1, h2.2, h3.1, h3.2⟩)
  | exact clash 2 3 (by decide) ((Or.inr ∘ Or.inr) ⟨h2.1, h2.2, h3.1, h3.2⟩)

```

end ProofPursuit.P4.Six

[illegible]

- exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 3, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨3, 4, 5, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 6, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 5, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 4, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩
- exact ⟨0, 1, 2, 3, by decide, by decide, by decide, by omega, by omega, by omega⟩

```

theorem seven_not_all_even (a : Fin 7 → Int) (m : Fin 7 → Nat)
  (hm : ∀ i, 0 < m i) (hd : DisjointClasses a m)
  (hn : ¬ ∃ i j : Fin 7, i < j ∧ 7 ≤ Nat.gcd (m i) (m j)) :
  ¬ ∀ i, 2 ∣ m i := by
intro he
obtain ⟨i, j, l, r, hij, hjl, hlr, haj, hal, har⟩ := four_same_parity a
let f : Fin 4 → Fin 7 := fun t => [i, j, l, r][t.val]'(by simp)
have hf : ∀ x y : Fin 4, f x = f y → x = y := by
  simp [Fin.forall_fin_succ, f]
  omega
have ha : ∀ t, a (f t) % 2 = a i % 2 := by
  simp [Fin.forall_fin_succ, f]
  exact ⟨haj.symm, hal.symm, har.symm⟩
let b : Fin 4 → Int := fun t => (a (f t) - a i % 2) / 2
let n : Fin 4 → Nat := fun t => m (f t) / 2
have hmEq (t : Fin 4) : 2 * n t = m (f t) := Nat.mul_div_cancel' (he (f t))
have haEq (t : Fin 4) : a (f t) = 2 * b t + a i % 2 := by
  have := ha t
  dsimp [b]
  omega
have hnp : ∀ t, 0 < n t := by
  intro t
  have := hmEq t
  have := hm (f t)
  omega
have hdis : DisjointClasses b n := by
  intro x y hxy hmeet
  obtain ⟨z, ⟨u, hu⟩, ⟨v, hv⟩⟩ := hmeet
  apply hd (f x) (f y) (fun h => hxy (hf x y h))
  refine ⟨2 * z + a i % 2, ?_, ?_⟩
  • refine ⟨u, ?_⟩
    have hmx : (m (f x) : Int) = 2 * (n x : Int) := by exact_mod_cast (hmEq x).symm
    have hax := haEq x
    grind
  • refine ⟨v, ?_⟩
    have hmy : (m (f y) : Int) = 2 * (n y : Int) := by exact_mod_cast (hmEq y).symm
    have hay := haEq y
    grind
obtain ⟨x, y, hxy, hg⟩ := C2.solution b n hnp hdis
have hxyn : f x ≠ f y := by intro h; have := hf x y h; omega
have hb := bounds hm hd hn hxyn
have hdiv : Nat.gcd (n x) (n y) = Nat.gcd (m (f x)) (m (f y)) / 2 :=

```

```

    Nat.gcd_div (he (f x)) (he (f y))
  omega

end ProofPursuit.P4

```

4.0.7 ProofPursuit/P4/Seven.lean

```

import ProofPursuit.P4.Parity

namespace ProofPursuit.P4.Seven
set_option maxHeartbeats 12000000

private theorem mod3_cases (b : Int) :
  b % 3 = 0 ∨ b % 3 = 1 ∨ b % 3 = 2 := by omega

private theorem allowed0 (b x : Int) (hb : b % 3 = 0) (ha : x % 3 ≠ b % 3) :
  x % 6 ∈ ([1, 2, 4, 5] : List Int) := by simp; omega

private theorem allowed1 (b x : Int) (hb : b % 3 = 1) (ha : x % 3 ≠ b % 3) :
  x % 6 ∈ ([0, 2, 3, 5] : List Int) := by simp; omega

private theorem allowed2 (b x : Int) (hb : b % 3 = 2) (ha : x % 3 ≠ b % 3) :
  x % 6 ∈ ([0, 1, 3, 4] : List Int) := by simp; omega

private theorem mod6_range (x : Int) :
  x % 6 ∈ ([0, 1, 2, 3, 4, 5] : List Int) := by simp; omega

private theorem mod5_range (x : Int) :
  x % 5 ∈ ([0, 1, 2, 3, 4] : List Int) := by simp; omega

private theorem seven_mod6_impossible (a : Fin 7 → Int)
  (h : ∀ i j : Fin 7, i ≠ j → a i % 6 ≠ a j % 6) : False := by
  let l : List Int := [a 0 % 6, a 1 % 6, a 2 % 6, a 3 % 6,
    a 4 % 6, a 5 % 6, a 6 % 6]
  have hnd : l.Nodup := by
    simp only [l, List.nodup_cons, List.mem_cons, List.mem_nil_iff,
      or_false, not_or]
  repeat' constructor
  all_goals first
  | simp
  | exact h 0 1 (by decide)
  | exact h 0 2 (by decide)
  | exact h 0 3 (by decide)
  | exact h 0 4 (by decide)
  | exact h 0 5 (by decide)
  | exact h 0 6 (by decide)
  | exact h 1 2 (by decide)
  | exact h 1 3 (by decide)
  | exact h 1 4 (by decide)
  | exact h 1 5 (by decide)
  | exact h 1 6 (by decide)
  | exact h 2 3 (by decide)
  | exact h 2 4 (by decide)
  | exact h 2 5 (by decide)
  | exact h 2 6 (by decide)
  | exact h 3 4 (by decide)
  | exact h 3 5 (by decide)
  | exact h 3 6 (by decide)
  | exact h 4 5 (by decide)
  | exact h 4 6 (by decide)
  | exact h 5 6 (by decide)
  have hs : l ⊆ ([0, 1, 2, 3, 4, 5] : List Int) := by

```

```

    intro x hx
    simp [l] at hx
    rcases hx with rfl | rfl | rfl | rfl | rfl | rfl | rfl <=>
    exact mod6_range _
have := hnd.length_le_of_subset hs
simp [l] at this

private theorem six_mod5_impossible (a : Fin 7 → Int)
  (h : ∀ i j : Fin 7, i ≠ j → a i % 5 ≠ a j % 5) : False := by
  let l : List Int := [a 0 % 5, a 1 % 5, a 2 % 5,
    a 3 % 5, a 4 % 5, a 5 % 5]
  have hnd : l.Nodup := by
    simp only [l, List.nodup_cons, List.mem_cons, List.mem_nil_iff,
      or_false, not_or]
  repeat' constructor
  all_goals first
  | simp
  | exact h 0 1 (by decide)
  | exact h 0 2 (by decide)
  | exact h 0 3 (by decide)
  | exact h 0 4 (by decide)
  | exact h 0 5 (by decide)
  | exact h 1 2 (by decide)
  | exact h 1 3 (by decide)
  | exact h 1 4 (by decide)
  | exact h 1 5 (by decide)
  | exact h 2 3 (by decide)
  | exact h 2 4 (by decide)
  | exact h 2 5 (by decide)
  | exact h 3 4 (by decide)
  | exact h 3 5 (by decide)
  | exact h 4 5 (by decide)
  have hs : l ⊆ ([0, 1, 2, 3, 4] : List Int) := by
    intro x hx
    simp [l] at hx
    rcases hx with rfl | rfl | rfl | rfl | rfl | rfl <=>
    exact mod5_range _
  have := hnd.length_le_of_subset hs
  simp [l] at this

theorem five_avoid (b x0 x1 x2 x3 x4 : Int)
  (ha0 : x0 % 3 ≠ b % 3)
  (ha1 : x1 % 3 ≠ b % 3)
  (ha2 : x2 % 3 ≠ b % 3)
  (ha3 : x3 % 3 ≠ b % 3)
  (ha4 : x4 % 3 ≠ b % 3)
  (hd01 : x0 % 6 ≠ x1 % 6)
  (hd02 : x0 % 6 ≠ x2 % 6)
  (hd03 : x0 % 6 ≠ x3 % 6)
  (hd04 : x0 % 6 ≠ x4 % 6)
  (hd12 : x1 % 6 ≠ x2 % 6)
  (hd13 : x1 % 6 ≠ x3 % 6)
  (hd14 : x1 % 6 ≠ x4 % 6)
  (hd23 : x2 % 6 ≠ x3 % 6)
  (hd24 : x2 % 6 ≠ x4 % 6)
  (hd34 : x3 % 6 ≠ x4 % 6)
  : False := by
  let l : List Int := [x0 % 6, x1 % 6, x2 % 6, x3 % 6, x4 % 6]
  have hnd : l.Nodup := by
    simp [l, List.nodup_cons, hd01, hd02, hd03, hd04,
      hd12, hd13, hd14, hd23, hd24, hd34]
  have hb := mod3_cases b
  rcases hb with hb | hb | hb

```

```

• have hs : l ⊆ ([1, 2, 4, 5] : List Int) := by
  intro y hy
  simp [l] at hy
  rcases hy with rfl | rfl | rfl | rfl | rfl
  • exact allowed0 b x0 hb ha0
  • exact allowed0 b x1 hb ha1
  • exact allowed0 b x2 hb ha2
  • exact allowed0 b x3 hb ha3
  • exact allowed0 b x4 hb ha4
  have := hnd.length_le_of_subset hs
  simp [l] at this
• have hs : l ⊆ ([0, 2, 3, 5] : List Int) := by
  intro y hy
  simp [l] at hy
  rcases hy with rfl | rfl | rfl | rfl | rfl
  • exact allowed1 b x0 hb ha0
  • exact allowed1 b x1 hb ha1
  • exact allowed1 b x2 hb ha2
  • exact allowed1 b x3 hb ha3
  • exact allowed1 b x4 hb ha4
  have := hnd.length_le_of_subset hs
  simp [l] at this
• have hs : l ⊆ ([0, 1, 3, 4] : List Int) := by
  intro y hy
  simp [l] at hy
  rcases hy with rfl | rfl | rfl | rfl | rfl
  • exact allowed2 b x0 hb ha0
  • exact allowed2 b x1 hb ha1
  • exact allowed2 b x2 hb ha2
  • exact allowed2 b x3 hb ha3
  • exact allowed2 b x4 hb ha4
  have := hnd.length_le_of_subset hs
  simp [l] at this

```

```
def target : Prop := Statement 7
```

```

theorem solution : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 7} (hij : i ≠ j) := bounds hm hd hn hij
  have ncommon (p : Nat) (hp : p = 2 ∨ p = 3 ∨ p = 5) : ¬ ∀ i, p ∣ m i := by
    rcases hp with rfl | rfl | rfl
    • exact seven_not_all_even a m hm hd hn
    • intro hc
      have hh (i j : Fin 7) (hij : i ≠ j) : Nat.gcd (m i) (m j) ∣ 6 := by
        have h := hb hij
        have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hc i) (hc j))
        have hx : Nat.gcd (m i) (m j) = 3 ∨ Nat.gcd (m i) (m j) = 6 := by omega
        rcases hx with hx | hx <=> simp [hx]
        exact seven_mod6_impossible a (fun i j hij =>
          residues_ne_of_gcd_dvd hd hij (hh i j hij))
    • intro hc
      have hh (i j : Fin 7) (hij : i ≠ j) : Nat.gcd (m i) (m j) ∣ 5 := by
        have h := hb hij
        have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd (hc i) (hc j))
        have hx : Nat.gcd (m i) (m j) = 5 := by omega
        simp [hx]
        exact six_mod5_impossible a (fun i j hij =>
          residues_ne_of_gcd_dvd hd hij (hh i j hij))
  have supports (i j : Fin 7) (hij : i ≠ j) :
    (2 ∣ m i ∧ 2 ∣ m j) ∨ (3 ∣ m i ∧ 3 ∣ m j) ∨ (5 ∣ m i ∧ 5 ∣ m j) := by
    have h := hb hij

```

```

have hl := Nat.gcd_dvd_left (m i) (m j)
have hr := Nat.gcd_dvd_right (m i) (m j)
have hx : Nat.gcd (m i) (m j) = 2 ∨ Nat.gcd (m i) (m j) = 3 ∨
  Nat.gcd (m i) (m j) = 4 ∨ Nat.gcd (m i) (m j) = 5 ∨ Nat.gcd (m i) (m j) = 6 := by omega
rcases hx with hx | hx | hx | hx | hx
· exact Or.inl ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩
· exact Or.inr (Or.inl ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩)
· exact Or.inl ⟨Nat.dvd_trans (by decide : 2 ∣ 4) (by simpa [hx] using hl),
  Nat.dvd_trans (by decide : 2 ∣ 4) (by simpa [hx] using hr)⟩
· exact Or.inr (Or.inr ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩)
· exact Or.inl ⟨Nat.dvd_trans (by decide : 2 ∣ 6) (by simpa [hx] using hl),
  Nat.dvd_trans (by decide : 2 ∣ 6) (by simpa [hx] using hr)⟩
have two (i : Fin 7) : (2 ∣ m i ∧ 3 ∣ m i) ∨ (2 ∣ m i ∧ 5 ∣ m i) ∨
  (3 ∣ m i ∧ 5 ∣ m i) := by
  have nc2 := ncommon 2 (by simp)
  have nc3 := ncommon 3 (by simp)
  have nc5 := ncommon 5 (by simp)
  by_cases h2 : 2 ∣ m i <=> by_cases h3 : 3 ∣ m i <=> by_cases h5 : 5 ∣ m i
  all_goals first
    | exact Or.inl ⟨h2, h3⟩
    | exact Or.inr (Or.inl ⟨h2, h5⟩)
    | exact Or.inr (Or.inr ⟨h3, h5⟩)
    | (ex falso; apply nc2; intro j; by_cases hij : i = j
      · simpa [← hij] using h2
      · rcases supports i j hij with h | h | h
        · exact h.2
        · exact False.elim (h3 h.1)
        · exact False.elim (h5 h.1))
    | (ex falso; apply nc3; intro j; by_cases hij : i = j
      · simpa [← hij] using h3
      · rcases supports i j hij with h | h | h
        · exact False.elim (h2 h.1)
        · exact h.2
        · exact False.elim (h5 h.1))
    | (ex falso; apply nc5; intro j; by_cases hij : i = j
      · simpa [← hij] using h5
      · rcases supports i j hij with h | h | h
        · exact False.elim (h2 h.1)
        · exact False.elim (h3 h.1)
        · exact h.2)
    | (have hx : ∃ j : Fin 7, i ≠ j := by
      by_cases h : i = 0
      · exact ⟨1, by omega⟩
      · exact ⟨0, h⟩
      obtain ⟨j, hj⟩ := hx
      rcases supports i j hj with h | h | h
      · exact False.elim (h2 h.1)
      · exact False.elim (h3 h.1)
      · exact False.elim (h5 h.1))
  obtain ⟨i, hi⟩ := Classical.not_forall.mp (ncommon 2 (by simp))
  obtain ⟨j, hj⟩ := Classical.not_forall.mp (ncommon 3 (by simp))
  have hi35 : 3 ∣ m i ∧ 5 ∣ m i := by
    rcases two i with h | h | h
    · exact False.elim (hi h.1)
    · exact False.elim (hi h.1)
    · exact h
  have hj25 : 2 ∣ m j ∧ 5 ∣ m j := by
    rcases two j with h | h | h
    · exact False.elim (hj h.2)
    · exact h
    · exact False.elim (hj h.1)
  have ht23 (t : Fin 7) (hti : t ≠ i) (htj : t ≠ j) : 2 ∣ m t ∧ 3 ∣ m t := by
    have h5 : ¬ 5 ∣ m t := by

```

```

intro h5
rcases two t with h | h | h
• have b := hb htj
  have x := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 hj25.1)
  have y := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h5 hj25.2)
  omega
• have b := hb htj
  have x := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 hj25.1)
  have y := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h5 hj25.2)
  omega
• have b := hb hti
  have x := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h.1 hi35.1)
  have y := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd h5 hi35.2)
  omega
rcases two t with h | h | h
• exact h
• exact False.elim (h5 h.2)
• exact False.elim (h5 h.2)
have avoid (t : Fin 7) (hti : t ≠ i) (htj : t ≠ j) : a t % 3 ≠ a i % 3 := by
have ht := ht23 t hti htj
have b := hb hti
have h3 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht.2 hi35.1)
have h6 : Nat.gcd (m t) (m i) ≠ 6 := by
  intro h
  apply hi
  exact Nat.dvd_trans (by decide : 2 | 6) (by simpa [h] using Nat.gcd_dvd_right (m t) (m i))
have hg : Nat.gcd (m t) (m i) = 3 := by omega
exact residues_ne hd hti hg
have different (t u : Fin 7) (htu : t ≠ u) (hti : t ≠ i) (htj : t ≠ j)
  (hui : u ≠ i) (huj : u ≠ j) : a t % 6 ≠ a u % 6 := by
have ht := ht23 t hti htj
have hu := ht23 u hui huj
have b := hb htu
have h2 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht.1 hu.1)
have h3 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht.2 hu.2)
have hg : Nat.gcd (m t) (m u) = 6 := by omega
exact residues_ne hd htu hg
have hic : i = 0 ∨ i = 1 ∨ i = 2 ∨ i = 3 ∨ i = 4 ∨ i = 5 ∨ i = 6 := by omega
have hjc : j = 0 ∨ j = 1 ∨ j = 2 ∨ j = 3 ∨ j = 4 ∨ j = 5 ∨ j = 6 := by omega
rcases hic with rfl | rfl | rfl | rfl | rfl | rfl | rfl <;>
rcases hjc with rfl | rfl | rfl | rfl | rfl | rfl | rfl
• exact hi hj25.1
• exact five_avoid (a 0) (a 2) (a 3) (a 4) (a 5) (a 6)
  (avoid 2 (by decide) (by decide))
  (avoid 3 (by decide) (by decide))
  (avoid 4 (by decide) (by decide))
  (avoid 5 (by decide) (by decide))
  (avoid 6 (by decide) (by decide))
  (different 2 3 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 2 4 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 2 5 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 2 6 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 3 4 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 3 5 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 3 6 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 4 5 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 4 6 (by decide) (by decide) (by decide) (by decide) (by decide))
  (different 5 6 (by decide) (by decide) (by decide) (by decide) (by decide))
• exact five_avoid (a 0) (a 1) (a 3) (a 4) (a 5) (a 6)
  (avoid 1 (by decide) (by decide))
  (avoid 3 (by decide) (by decide))
  (avoid 4 (by decide) (by decide))
  (avoid 5 (by decide) (by decide))

```

[illegible]

[illegible]

[illegible]


```

    (different 0 1 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 5 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 3 5 (by decide) (by decide) (by decide) (by decide) (by decide))
  · exact five_avoid (a 6) (a 0) (a 1) (a 2) (a 3) (a 4)
    (avoid 0 (by decide) (by decide))
    (avoid 1 (by decide) (by decide))
    (avoid 2 (by decide) (by decide))
    (avoid 3 (by decide) (by decide))
    (avoid 4 (by decide) (by decide))
    (different 0 1 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 0 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 2 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 1 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 3 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 2 4 (by decide) (by decide) (by decide) (by decide) (by decide))
    (different 3 4 (by decide) (by decide) (by decide) (by decide) (by decide))
  · exact hi hj25.1

end ProofPursuit.P4.Seven

```

4.0.8 ProofPursuit/P4/Eight.lean

```

import ProofPursuit.P4.Basic

namespace ProofPursuit.P4.Eight

set_option maxHeartbeats 12000000

private def omitIndex (t : Fin 8) (s : Fin 7) : Fin 8 :=
  (if s.val < t.val then s.val else s.val + 1, by
    by_cases h : s.val < t.val <=> simp [h] <=> omega)

private theorem omitIndex_ne (t : Fin 8) (s : Fin 7) : omitIndex t s ≠ t := by
  intro h
  have hv := congrArg Fin.val h
  simp only [omitIndex] at hv
  by_cases hs : s.val < t.val <=> simp [hs] at hv <=> omega

private theorem omitIndex_injective (t : Fin 8) {s u : Fin 7}
  (h : omitIndex t s = omitIndex t u) : s = u := by
  have hv := congrArg Fin.val h
  simp only [omitIndex] at hv
  by_cases hs : s.val < t.val <=> by_cases hu : u.val < t.val <=>
    simp [hs, hu] at hv <=> apply Fin.ext <=> omega

private theorem omitIndex_surjective (t i : Fin 8) (hit : i ≠ t) :
  ∃ s : Fin 7, omitIndex t s = i := by
  have hv : i.val ≠ t.val := by intro h; exact hit (Fin.ext h)
  by_cases h : i.val < t.val
  · refine ⟨i.val, by omega, ?_⟩
    apply Fin.ext
    simp [omitIndex, h]

```

```

· refine ⟨⟨i.val - 1, by omega⟩, ?_⟩
  apply Fin.ext
  simp [omitIndex, show ¬ i.val - 1 < t.val by omega]
  omega

def target : Prop := Statement 8

theorem step (h7 : Statement 7) : target := by
  intro a m hm hd
  apply Classical.byContradiction
  intro hn
  have hb {i j : Fin 8} (hij : i ≠ j) := bounds hm hd hn hij
  have sevenPair (t : Fin 8) :
    ∃ i j : Fin 8, i ≠ j ∧ i ≠ t ∧ j ≠ t ∧ Nat.gcd (m i) (m j) = 7 := by
    let a' : Fin 7 → Int := fun s => a (omitIndex t s)
    let m' : Fin 7 → Nat := fun s => m (omitIndex t s)
    have hm' : ∀ s, 0 < m' s := by intro s; exact hm _
    have hd' : DisjointClasses a' m' := by
      intro s u hsu
      exact hd (omitIndex t s) (omitIndex t u)
        (fun h => hsu (omitIndex_injective t h))
    obtain ⟨s, u, hsu, hge⟩ := h7 a' m' hm' hd'
    have hne : omitIndex t s ≠ omitIndex t u := by
      intro h
      have := omitIndex_injective t h
      omega
    refine ⟨omitIndex t s, omitIndex t u, hne, omitIndex_ne t s,
      omitIndex_ne t u, ?_⟩
    have hlt := (hb hne).2
    dsimp [m'] at hge
    omega
  have gcdSeven (x y : Fin 8) (hxy : x ≠ y)
    (hx : 7 ∣ m x) (hy : 7 ∣ m y) : Nat.gcd (m x) (m y) = 7 := by
    have h := hb hxy
    have hz := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd hx hy)
    omega
  have noAllSeven : ¬ ∀ x : Fin 8, 7 ∣ m x := by
    intro hall
    have hres (x y : Fin 8) (hxy : x ≠ y) : a x % 7 ≠ a y % 7 :=
      residues_ne hd hxy (gcdSeven x y hxy (hall x) (hall y))
    let l : List Int := List.ofFn (fun x : Fin 8 => a x % 7)
    have hnodup : l.Nodup := by
      rw [List.nodup_iff_eq_of_getElem_eq]
      intro x y hx hy heq
      have hx8 : x < 8 := by simp [l] using hx
      have hy8 : y < 8 := by simp [l] using hy
      apply Classical.byContradiction
      intro hxy
      have hfi : ⟨x, hx8⟩ : Fin 8 ≠ ⟨y, hy8⟩ := by
        intro h
        apply hxy
        exact congrArg Fin.val h
      have hne := hres ⟨x, hx8⟩ ⟨y, hy8⟩ hfi
      apply hne
      simp only [l, List.getElem_ofFn] using heq
    have hsub : l ⊆ ([0, 1, 2, 3, 4, 5, 6] : List Int) := by
      intro x hx
      obtain ⟨i, rfl⟩ := List.mem_ofFn.mp hx
      have h0 := Int.emod_nonneg (a i) (by decide : (7 : Int) ≠ 0)
      have h1 := Int.emod_lt_of_pos (a i) (by decide : (0 : Int) < 7)
      simp only [List.mem_cons, List.mem_nil_iff, or_false]
      omega
    have hc := hnodup.length_le_of_subset hsub

```

```

simp [l] at hc
obtain ⟨i, j, hij, _, _, hij7⟩ := sevenPair 0
have hi7 : 7 ∣ m i := by
  rw [← hij7]
  exact Nat.gcd_dvd_left (m i) (m j)
have hj7 : 7 ∣ m j := by
  rw [← hij7]
  exact Nat.gcd_dvd_right (m i) (m j)
have third : ∃ k : Fin 8, k ≠ i ∧ k ≠ j ∧ 7 ∣ m k := by
  obtain ⟨p, q, hpq, hpi, hqi, hpq7⟩ := sevenPair i
  by_cases hpj : p = j
  · refine ⟨q, hqi, ?_, ?_⟩
    · exact fun h => hpq (hpj.trans h.symm)
    · rw [← hpq7]
      exact Nat.gcd_dvd_right (m p) (m q)
  · refine ⟨p, hpi, hpj, ?_⟩
    rw [← hpq7]
    exact Nat.gcd_dvd_left (m p) (m q)
obtain ⟨k, hki, hkj, hk7⟩ := third
obtain ⟨t, ht7⟩ := Classical.not_forall.mp noAllSeven
have hti : t ≠ i := by intro h; exact ht7 (h ▸ hi7)
have htj : t ≠ j := by intro h; exact ht7 (h ▸ hj7)
have htk : t ≠ k := by intro h; exact ht7 (h ▸ hk7)
have noCommon (x y : Fin 8) (hxy : x ≠ y)
  (hx : 7 ∣ m x) (hy : 7 ∣ m y) (p : Nat)
  (hp : p = 2 ∨ p = 3 ∨ p = 5) : ¬(p ∣ m x ∧ p ∣ m y) := by
  intro h
  have hdvd : p ∣ 7 := by
    rw [← gcdSeven x y hxy hx hy]
    exact Nat.dvd_gcd h.1 h.2
  rcases hp with rfl | rfl | rfl
  · exact (by decide : ¬ 2 ∣ 7) hdvd
  · exact (by decide : ¬ 3 ∣ 7) hdvd
  · exact (by decide : ¬ 5 ∣ 7) hdvd
have sharedSmall (x y : Fin 8) (hxy : x ≠ y) (hy : ¬ 7 ∣ m y) :
  (2 ∣ m x ∧ 2 ∣ m y) ∨
  (3 ∣ m x ∧ 3 ∣ m y) ∨
  (5 ∣ m x ∧ 5 ∣ m y) := by
  have h := hb hxy
  have hl := Nat.gcd_dvd_left (m x) (m y)
  have hr := Nat.gcd_dvd_right (m x) (m y)
  have hne : Nat.gcd (m x) (m y) ≠ 7 := by
    intro heq
    apply hy
    rw [← heq]
    exact hr
  have hx : Nat.gcd (m x) (m y) = 2 ∨ Nat.gcd (m x) (m y) = 3 ∨
    Nat.gcd (m x) (m y) = 4 ∨ Nat.gcd (m x) (m y) = 5 ∨
    Nat.gcd (m x) (m y) = 6 := by omega
  rcases hx with hx | hx | hx | hx | hx
  · exact Or.inl ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩
  · exact Or.inr (Or.inl ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩)
  · exact Or.inl ⟨Nat.dvd_trans (by decide : 2 ∣ 4) (by simpa [hx] using hl),
    Nat.dvd_trans (by decide : 2 ∣ 4) (by simpa [hx] using hr)⟩
  · exact Or.inr (Or.inr ⟨by simpa [hx] using hl, by simpa [hx] using hr⟩)
  · exact Or.inl ⟨Nat.dvd_trans (by decide : 2 ∣ 6) (by simpa [hx] using hl),
    Nat.dvd_trans (by decide : 2 ∣ 6) (by simpa [hx] using hr)⟩
have h_it := sharedSmall i t hti.symm ht7
have h_jt := sharedSmall j t htj.symm ht7
have h_kt := sharedSmall k t htk.symm ht7
have h_ij2 := noCommon i j hij hi7 hj7 2 (Or.inl rfl)
have h_ij3 := noCommon i j hij hi7 hj7 3 (Or.inr (Or.inl rfl))
have h_ij5 := noCommon i j hij hi7 hj7 5 (Or.inr (Or.inr rfl))

```

```

have h_ik2 := noCommon i k hki.symm hi7 hk7 2 (Or.inl rfl)
have h_ik3 := noCommon i k hki.symm hi7 hk7 3 (Or.inr (Or.inl rfl))
have h_ik5 := noCommon i k hki.symm hi7 hk7 5 (Or.inr (Or.inr rfl))
have h_jk2 := noCommon j k hkj.symm hj7 hk7 2 (Or.inl rfl)
have h_jk3 := noCommon j k hkj.symm hj7 hk7 3 (Or.inr (Or.inl rfl))
have h_jk5 := noCommon j k hkj.symm hj7 hk7 5 (Or.inr (Or.inr rfl))
have ht235 : 2 | m t  $\wedge$  3 | m t  $\wedge$  5 | m t := by grind
have fifth :  $\exists u : \text{Fin } 8, u \neq i \wedge u \neq j \wedge u \neq k \wedge u \neq t :=$  by
  apply Classical.byContradiction
  intro h
  have h0 :  $\neg ((0 : \text{Fin } 8) \neq i \wedge 0 \neq j \wedge 0 \neq k \wedge 0 \neq t) :=$  by
    intro hh; exact h ⟨0, hh⟩
  have h1 :  $\neg ((1 : \text{Fin } 8) \neq i \wedge 1 \neq j \wedge 1 \neq k \wedge 1 \neq t) :=$  by
    intro hh; exact h ⟨1, hh⟩
  have h2 :  $\neg ((2 : \text{Fin } 8) \neq i \wedge 2 \neq j \wedge 2 \neq k \wedge 2 \neq t) :=$  by
    intro hh; exact h ⟨2, hh⟩
  have h3 :  $\neg ((3 : \text{Fin } 8) \neq i \wedge 3 \neq j \wedge 3 \neq k \wedge 3 \neq t) :=$  by
    intro hh; exact h ⟨3, hh⟩
  have h4 :  $\neg ((4 : \text{Fin } 8) \neq i \wedge 4 \neq j \wedge 4 \neq k \wedge 4 \neq t) :=$  by
    intro hh; exact h ⟨4, hh⟩
  omega
obtain ⟨u, hui, huj, huk, hut⟩ := fifth
have hu7 :  $\neg 7 | m u :=$  by
  intro hu7
  have h_ut := sharedSmall u t hut ht7
  have h_ui2 := noCommon u i hui hu7 hi7 2 (Or.inl rfl)
  have h_ui3 := noCommon u i hui hu7 hi7 3 (Or.inr (Or.inl rfl))
  have h_ui5 := noCommon u i hui hu7 hi7 5 (Or.inr (Or.inr rfl))
  have h_uj2 := noCommon u j huj hu7 hj7 2 (Or.inl rfl)
  have h_uj3 := noCommon u j huj hu7 hj7 3 (Or.inr (Or.inl rfl))
  have h_uj5 := noCommon u j huj hu7 hj7 5 (Or.inr (Or.inr rfl))
  have h_uk2 := noCommon u k huk hu7 hk7 2 (Or.inl rfl)
  have h_uk3 := noCommon u k huk hu7 hk7 3 (Or.inr (Or.inl rfl))
  have h_uk5 := noCommon u k huk hu7 hk7 5 (Or.inr (Or.inr rfl))
  grind
have h_iu := sharedSmall i u hui.symm hu7
have h_ju := sharedSmall j u huj.symm hu7
have h_ku := sharedSmall k u huk.symm hu7
have hu235 : 2 | m u  $\wedge$  3 | m u  $\wedge$  5 | m u := by grind
have hbound := hb hut.symm
have hg2 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht235.1 hu235.1)
have hg3 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht235.2.1 hu235.2.1)
have hg5 := Nat.mod_eq_zero_of_dvd (Nat.dvd_gcd ht235.2.2 hu235.2.2)
omega

```

end ProofPursuit.P4.Eight

4.0.9 ProofPursuit/P4/Progress.lean

```

import ProofPursuit.P4.C1
import ProofPursuit.P4.C2
import ProofPursuit.P4.Five
import ProofPursuit.P4.Six
import ProofPursuit.P4.Seven
import ProofPursuit.P4.Eight

```

namespace ProofPursuit.P4.Progress

```

/-- The k = 2 base case, included to make the certified range contiguous. -/
theorem two : Statement 2 := by
  intro a m hm hd
  apply Classical.byContradiction

```

```

intro hn
have := bounds hm hd hn (by decide : (0 : Fin 2) ≠ 1)
omega

/-- The complete contiguous range required by P4 C3. -/
def target : Prop := ∀ k : Nat, 2 ≤ k → k ≤ 8 → Statement k

theorem solution : target := by
  intro k hlo hhi
  have hk : k = 2 ∨ k = 3 ∨ k = 4 ∨ k = 5 ∨ k = 6 ∨ k = 7 ∨ k = 8 := by omega
  rcases hk with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  · exact two
  · exact C1.solution
  · exact C2.solution
  · exact Five.solution
  · exact Six.solution
  · exact Seven.solution
  · exact Eight.step Seven.solution

#print axioms solution
#print axioms C1.solution
#print axioms C2.solution
#print axioms Five.solution
#print axioms Six.solution
#print axioms Seven.solution
#print axioms Eight.step

end ProofPursuit.P4.Progress

```

4.0.10 ProofPursuit/P4/C3.lean

```

import ProofPursuit.P4.Progress

namespace ProofPursuit.P4.C3

/-- Every size from 2 through 8, with no assumption about smaller cases. -/
def target : Prop := ∀ k : Nat, 2 ≤ k → k ≤ 8 → Statement k

theorem solution : target := Progress.solution

#print axioms solution

end ProofPursuit.P4.C3

```